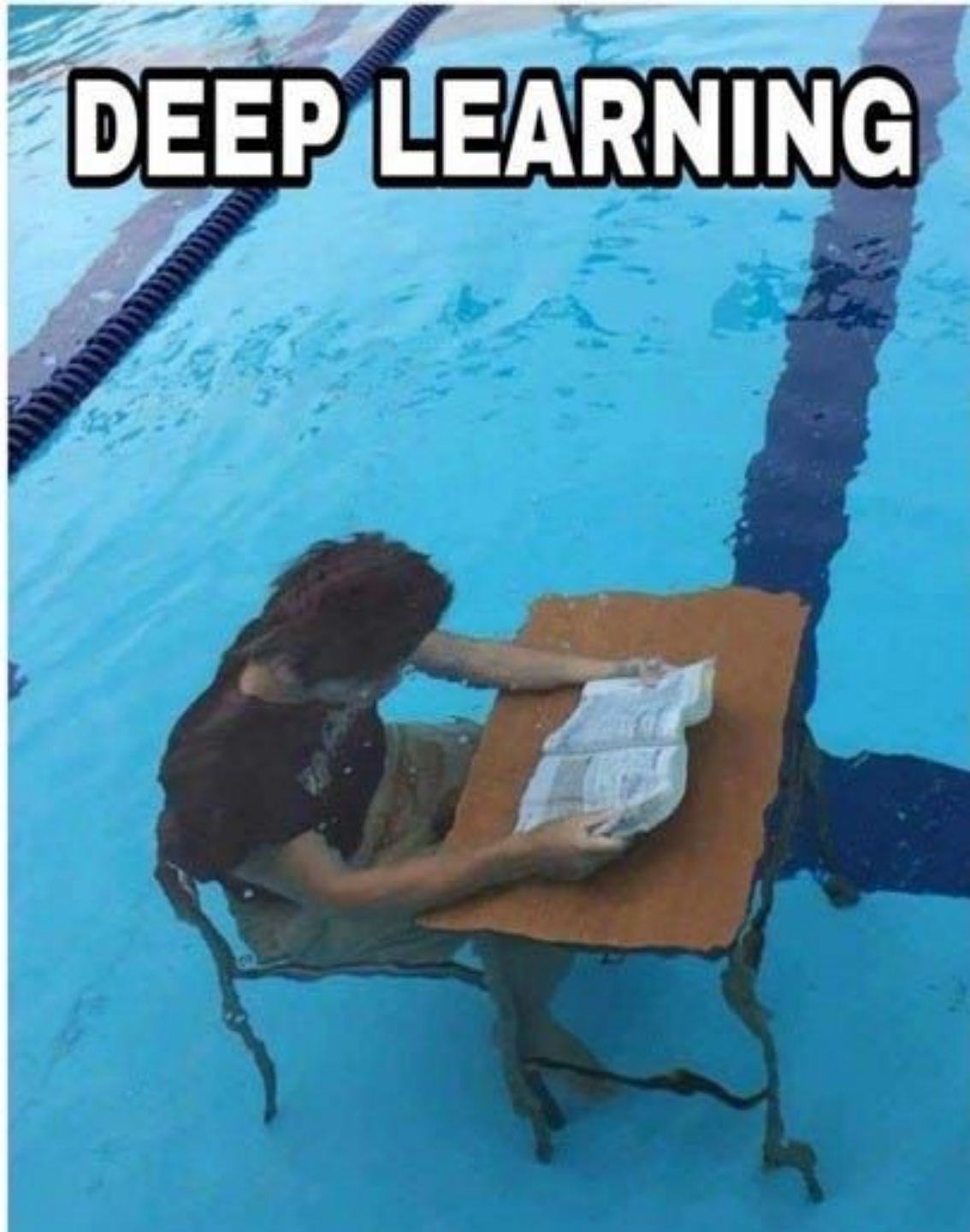


# MACHINE LEARNING



(aprendiendo machín)

# DEEP LEARNING



CLASE 5

# Regresión y Clasificación

*Tres modelos para predecir números.  
Cuatro para predecir categorías.*

**Minería de Datos** · Ing. de Software 9° sem · UPCh · 2026A

Prof. Ramsés Camas Nájera

# Objetivos de aprendizaje

- 1 **Distinguir** el problema de regresión del de clasificación, en términos del tipo de salida y de la función de pérdida.
- 2 **Construir** intuición sobre 3 modelos de regresión: lineal, regularizada (Ridge / Lasso) y Random Forest.
- 3 **Construir** intuición sobre 4 modelos de clasificación: árbol de decisión, Naïve Bayes, k-NN y regresión logística.
- 4 **Evaluar** el desempeño con métricas adecuadas: MSE /  $R^2$  para regresión; precisión, recall, F1, ROC-AUC para clasificación.
- 5 **Justificar** la elección de un modelo en función del dataset, la métrica y el costo de los errores.

# Agenda

## PARTE 1

### Regresión

*Predecir un valor continuo*

- Regresión lineal
- Ridge (L2)
- Lasso (L1)

## PARTE 2

### Clasificación

*Predecir una categoría*

- Árbol de decisión
- Naïve Bayes
- k-NN
- Regresión logística

## PARTE 3

### Evaluación + Actividad

*Medir, comparar, decidir*

- Métricas y validación
- Actividad sobre Telco Churn
- Entrega: notebook .ipynb

# ¿Qué es regresión?

*Aprender una función  $f$  que mapea atributos de entrada a un valor numérico continuo.*



## Ejemplos:

predecir el precio de una casa · estimar TotalCharges de un cliente Telco · forecast del ingreso anual · pronóstico de demanda mensual

# Regresión vs Clasificación

## REGRESIÓN

### Salida

Número real (continuo)

### Ejemplo

TotalCharges = \$1,840.50

### Pérdida típica

MSE, MAE

### Métrica

RMSE,  $R^2$

### Frontera

Curva ajustada a los datos

## CLASIFICACIÓN

### Salida

Categoría (discreta)

### Ejemplo

Churn = Sí / No

### Pérdida típica

Cross-entropy, 0/1 loss

### Métrica

Accuracy, F1, AUC

### Frontera

Superficie que separa clases



# Notación que usaremos

**X**

**Matriz de atributos · forma (n, p)**

*n = tuplas, p = atributos por tupla*

**$x_i$**

**Vector de la tupla i**

*$x_i = (x_{i1}, x_{i2}, \dots, x_{ip})$*

**$y_i$**

**Valor real (observado)**

*lo que sí ocurrió en el dataset*

**$\hat{y}_i$**

**Valor predicho por el modelo**

*lo que el modelo cree que ocurrirá*

**w, b**

**Parámetros aprendidos**

*pesos (w) y sesgo (b)*

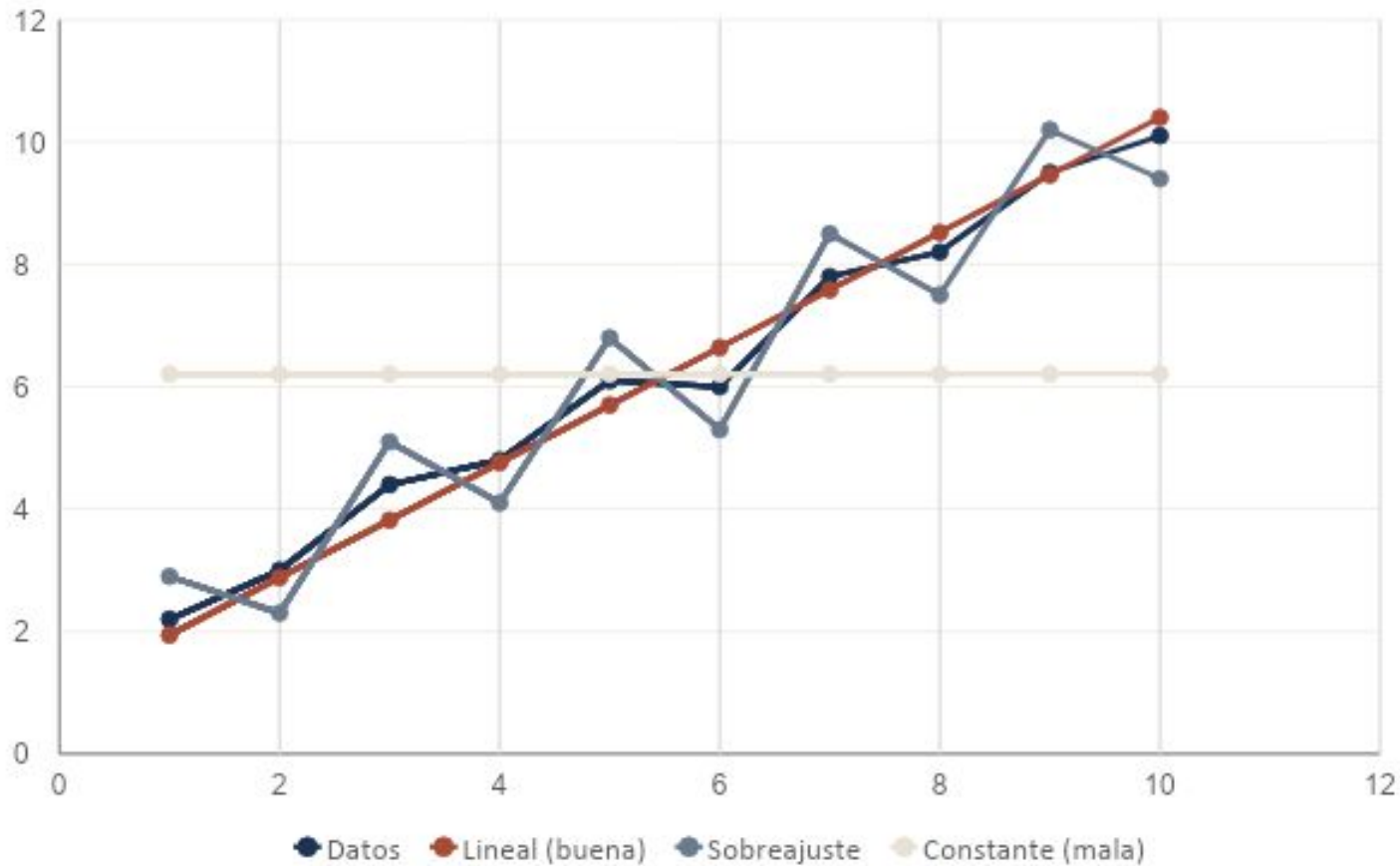
**L**

**Función de pérdida**

*mide qué tan equivocado está el modelo*



# ¿Qué función mejor mapea X a y?



## Tres candidatos:

- Constante (subajusta)
- **Lineal (generaliza)**
- Curva enredada (sobreajusta)

*El modelo correcto es el que captura la señal sin memorizar el ruido.*

MODELO 1

# Regresión Lineal

*La línea recta que minimiza el error cuadrático.*

# Una combinación ponderada

$$\hat{y} = w_1x_1 + w_2x_2 + \dots + w_jx_j + b$$

$\hat{y}$

**Predicción**

*lo que queremos estimar*

$w_j$

**Peso**

*qué tan importante es  $x_j$*

$x_j$

**Atributo**

*valor  $j$  de la observación*

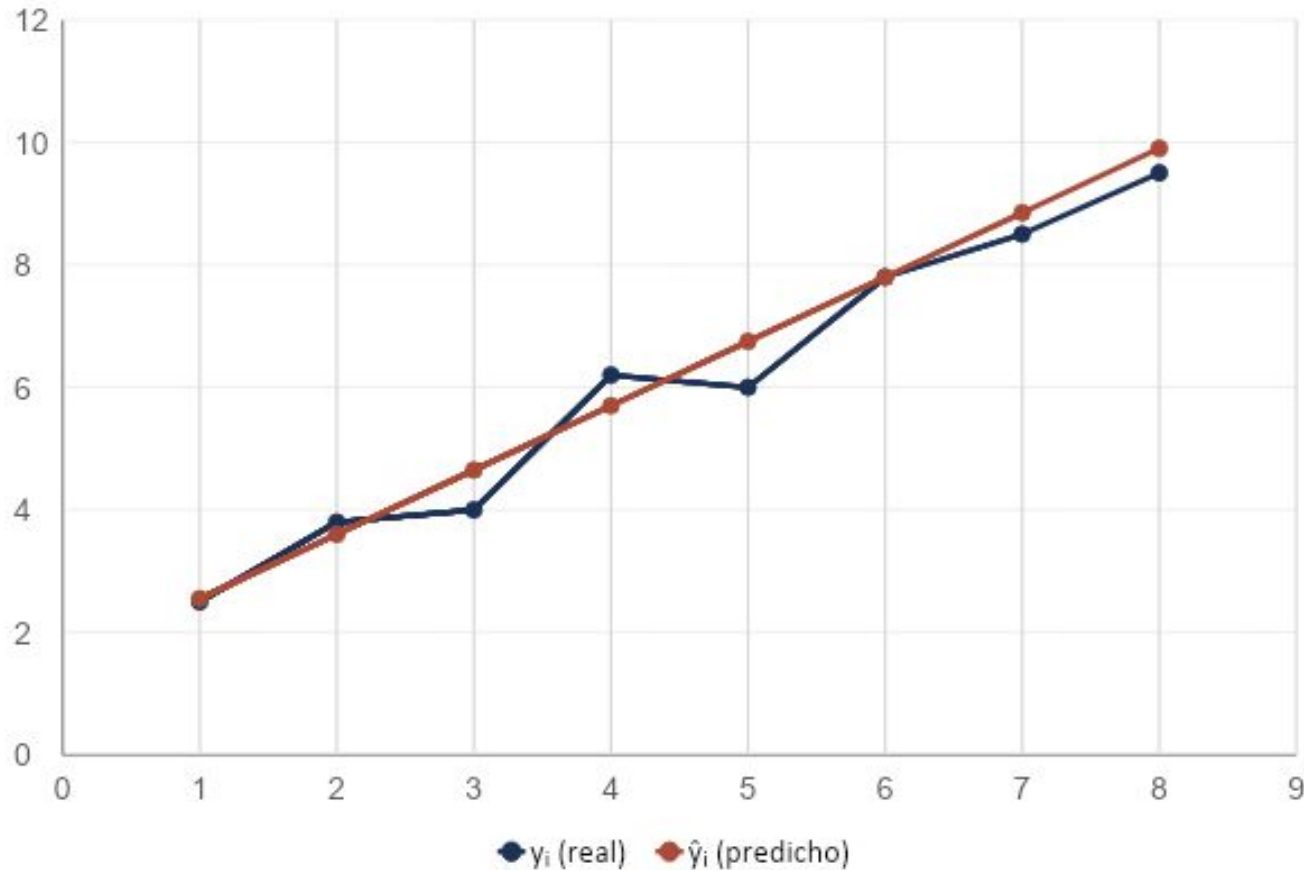
$b$

**Sesgo**

*desplaza la línea verticalmente*

Cada peso  $w_j$  mide cuánto cambia la predicción cuando  $x_j$  sube en una unidad. El sesgo  $b$  ajusta la línea para que pase por la nube de puntos.

# Mínimos cuadrados (Least Squares)



**Objetivo:**

$$RSS = \sum_i (y_i - \hat{y}_i)^2$$

Suma de los cuadrados de las distancias verticales entre cada punto real ( $y_i$ ) y la línea predicha ( $\hat{y}_i$ ).

*Elevar al cuadrado castiga errores grandes desproporcionadamente — es lo que vuelve a la regresión lineal sensible a outliers.*

# Ejemplo: 4 puntos

## Datos de entrenamiento

| i | $x_i$ | $y_i$ |
|---|-------|-------|
| 1 | 1     | 4     |
| 2 | 3     | 10    |
| 3 | 5     | 14    |
| 4 | 7     | 16    |

## Resolviendo

$$\bar{y} = (4+10+14+16) / 4 = 11$$

$$w = \frac{\sum x_i(y_i - \bar{y})}{[\sum x_i^2 - (1/n)(\sum x_i)^2]}$$
$$= 40 / (84 - 64) = \mathbf{2}$$

$$b = \bar{y} - w \cdot \bar{x} = 11 - 2 \cdot 4 = \mathbf{3}$$

**Modelo resultante:  $\hat{y} = 2x + 3$**

# Métricas de regresión

## MSE

*Mean Squared Error*

$$(1/n) \sum (y_i - \hat{y}_i)^2$$

Penaliza outliers fuerte.

## RMSE

*Root Mean Squared Error*

$$\sqrt{\text{MSE}}$$

En las mismas unidades que y.

## MAE

*Mean Absolute Error*

$$(1/n) \sum |y_i - \hat{y}_i|$$

Robusto a outliers.

## R<sup>2</sup>

*Coeficiente de determinación*

$$1 - (\text{RSS} / \text{TSS})$$

Fracción de varianza explicada  
(0–1).

*Regla práctica: reporta RMSE para magnitud del error y R<sup>2</sup> para qué tan bien explica la varianza.*

# Tres limitaciones que debes conocer

1

## Outliers

Como minimiza errores al cuadrado, un punto muy alejado puede inclinar toda la línea. Robust regression o transformaciones logarítmicas ayudan.

2

## Multicolinealidad

Si dos atributos están altamente correlacionados (p. ej., tenure y TotalCharges en Telco), los pesos se vuelven inestables. Mitiga con Ridge.

3

## No linealidad

Si la relación real entre  $X$  y  $y$  es curva, una recta jamás la captará. Necesitas modelos no lineales: árboles, polinomios o splines.



## MODELO 2

# Ridge y Lasso

*Regresión lineal regularizada: pesos más pequeños, modelos más robustos.*

# El modelo memoriza, no aprende

## Síntomas de overfitting:

- Excelente desempeño en train, pobre en test.
- Pesos  $w$  con magnitudes absurdamente grandes.
- Modelo extremadamente sensible al ruido de un atributo individual.
- Si quitas una sola observación, el modelo cambia mucho.

## Idea de la regularización

*Penalizar pesos grandes durante el entrenamiento.*

Pérdida regularizada:

$$L(w) = \text{RSS} + \alpha \cdot \text{penalización}(w)$$

$\alpha$  controla cuánto castigamos.  $\alpha$  grande  $\rightarrow$  pesos muy pequeños (bias alto).  $\alpha$  pequeño  $\rightarrow$  casi sin regularizar (varianza alta).

# Penalización L2: encoge los pesos

$$L(w) = \sum (y_i - \hat{y}_i)^2 + \alpha \cdot \sum w^2$$

## QUÉ HACE

- Reduce todos los pesos proporcionalmente.
- Nunca los hace exactamente cero.
- Estabiliza el modelo ante multicolinealidad.

## CUÁNDO USARLO

- Tienes muchos atributos correlacionados.
- Quieres conservar todos los atributos pero suavizados.
- Tu prioridad es estabilidad de las predicciones.

# Penalización L1: pone pesos en cero

$$L(w) = \sum (y_i - \hat{y}_i)^2 + \alpha \cdot \sum |w_j|$$

## QUÉ HACE

- Pesos de atributos irrelevantes  $\rightarrow$  exactamente 0.
- Realiza selección de atributos automáticamente.
- Produce modelos más interpretables y dispersos.

## CUÁNDO USARLO

- Sospechas que solo unos pocos atributos importan.
- Quieres un modelo interpretable y delgado.
- Tienes muchos más atributos que tuplas ( $p > n$ ).

# De continuo a categórico

Antes (regresión):

$$\hat{y} = 1840.50$$

*Salida = un número.  
Cualquier número real.*

Error = distancia  $(y_i - \hat{y}_i)^2$

Ahora (clasificación):

$$\hat{y} = \text{Churn}$$

*Salida = una etiqueta.  
De un conjunto finito.*

Error = ¿acertó o falló?

# Aplicaciones reales

## TELCO

¿Este cliente va a cancelar el servicio?

**Churn:** sí / no

## SALUD

¿Esta resonancia muestra signos tempranos?

**Diagnóstico:** sí / no

## BANCO

¿Aprobar este crédito?

**Riesgo:** bajo / alto

## SEGURIDAD

¿Este tráfico de red es un ataque?

**Intrusión:** sí / no

## MARKETING

¿Este review es real o spam?

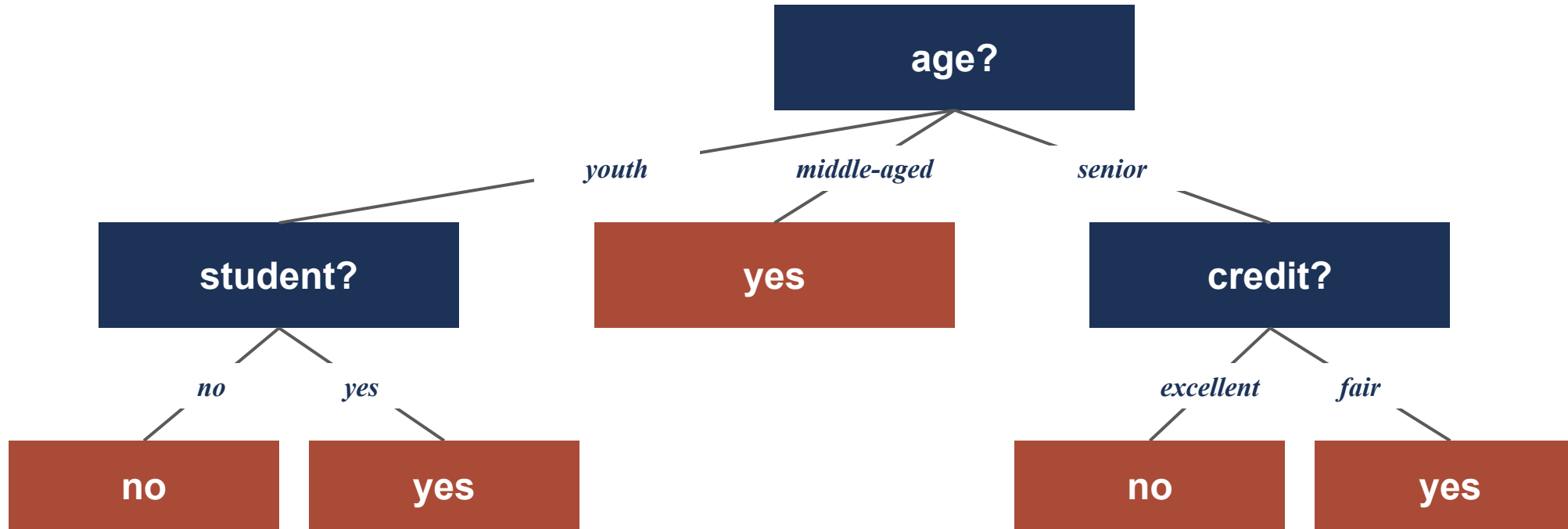
**Review:** real / fake

## MANUFACTURA

¿Esta pieza tiene defecto visual?

**Pieza:** OK / defectuosa

# Una secuencia de preguntas sí/no



*Internal (azul) = test sobre un atributo · Hoja (terracota) = clase predicha (¿buys\_computer?)*

# Algoritmo top-down divide-and-conquer

- 1 Empieza con todas las tuplas en el nodo raíz.
- 2 Elige el atributo que mejor separa las clases (information gain o Gini).
- 3 Divide las tuplas según los valores de ese atributo en ramas.
- 4 Repite recursivamente en cada rama.
- 5 Detente cuando: todas las tuplas son de una clase · no quedan atributos · partición vacía.

*ID3, C4.5 y CART son las tres familias clásicas que implementan esta misma idea con criterios distintos.*



# Information Gain vs Gini impurity

## INFORMATION GAIN

$$\text{Info}(D) = - \sum p_i \cdot \log_2(p_i)$$

- Usado por ID3 y C4.5.
- Mide reducción de entropía.
- Permite splits multiway.
- Sesgado hacia atributos con muchos valores.

## GINI IMPURITY

$$\text{Gini}(D) = 1 - \sum p_i^2$$

- Usado por CART.
- Mide probabilidad de mala clasificación.
- Solo splits binarios.
- Más eficiente computacionalmente (sin log).

# Overfitting y pruning

## Síntoma:

Un árbol sin restricciones memoriza el ruido de los datos de entrenamiento — perfecta exactitud en train, mala en test.

### PRE-PRUNING

#### Detén el crecimiento antes.

Limita `max_depth`, exige mínimo de tuplas por nodo, o umbral de ganancia mínima. Rápido pero arriesga subajustar.

### POST-PRUNING

#### Deja crecer, después corta.

Construye el árbol completo y luego elimina ramas usando un set de validación o cost-complexity. Más lento pero más confiable.

# Ventajas que no han envejecido

## Interpretable

Cada predicción es trazable como una cadena de preguntas. Un humano puede explicarla.

## Sin escalado

Funciona con atributos en distintas escalas (edad en años, monto en pesos) sin normalizar.

## Mezcla de tipos

Acepta atributos numéricos y categóricos en el mismo modelo sin trucos.

## Tolera missing

Soporta valores faltantes con strategies como surrogate splits.

## Captura no-linealidad

La regla de decisión cambia en cada split — no asume linealidad.

## Base de ensembles

Es el ladrillo de Random Forest, AdaBoost, XGBoost — los modelos campeones en tabular.

# Teorema de Bayes aplicado

$$P(C | X) = [ P(X | C) \cdot P(C) ] / P(X)$$

**POSTERIOR** **$P(C | X)$** 

Probabilidad de clase  
C dado X

**LIKELIHOOD** **$P(X | C)$** 

Qué tan probable es X  
bajo C

**PRIOR** **$P(C)$** 

Prevalencia general de  
la clase C

**EVIDENCIA** **$P(X)$** 

Probabilidad marginal  
de X

*El clasificador asigna a X la clase C con la mayor  $P(C | X)$ .*

# Independencia condicional

$$P(X | C) \approx P(x_1|C) \cdot P(x_2|C) \cdot \dots \cdot P(x_n | C)$$

## LA SUPOSICIÓN

**Los atributos son independientes entre sí, dado el valor de la clase.**

*Ej.: dado que un email es spam, la palabra 'gratis' es independiente de la palabra 'click'.*

## LA REALIDAD

**Casi nunca es cierto. Los atributos suelen estar correlacionados.**

*Y aun así Naïve Bayes funciona sorprendentemente bien. La suposición es errónea pero útil — sacrifica precisión teórica por velocidad y simplicidad.*

# Casos en los que es la mejor elección

## Clasificación de texto

Spam, sentimiento, categorización de noticias. La independencia entre palabras importa menos en la práctica.

## Datasets muy pequeños

Con decenas de ejemplos ya entrena. Sus probabilidades necesitan menos datos que un árbol o una red.

## Baseline rápido

Entrena en un solo pase por los datos. Útil como punto de partida antes de modelos más caros.

## Predicciones probabilísticas

Devuelve  $P(\text{clase} \mid X)$  calibrada — útil si te importa la confianza, no solo la etiqueta.

# Predecir buys\_computer a mano

## Datos (5 tuplas)

| edad  | ingreso | buys? |
|-------|---------|-------|
| joven | alto    | no    |
| joven | bajo    | sí    |
| joven | bajo    | sí    |
| mayor | alto    | sí    |
| mayor | bajo    | no    |

Predecir clase de:

**X = (joven, alto)**

## Cálculo

### Priors:

$$P(\text{sí}) = 3/5 = 0.60 \quad P(\text{no}) = 2/5 = 0.40$$

### Likelihoods:

$$P(\text{joven} \mid \text{sí}) = 2/3 \quad P(\text{joven} \mid \text{no}) = 1/2$$

$$P(\text{alto} \mid \text{sí}) = 1/3 \quad P(\text{alto} \mid \text{no}) = 1/2$$

### Posteriores ( $\propto$ ):

$$P(\text{sí} \mid X) \propto 0.60 \cdot 2/3 \cdot 1/3 = \mathbf{0.133}$$

$$P(\text{no} \mid X) \propto 0.40 \cdot 1/2 \cdot 1/2 = \mathbf{0.100}$$

**Predicción: sí compra (0.133 > 0.100)**

# Aprendizaje perezoso: dime con quién andas

*Para clasificar un punto nuevo, busca sus  $k$  vecinos más cercanos en el espacio de atributos y vota por la clase mayoritaria.*

## Sin entrenamiento

No construye un modelo. Solo guarda los datos. Toda la computación ocurre en la predicción.

## Distancia importa

Euclidiana, Manhattan, coseno. La elección de métrica cambia la frontera de decisión.

## Sensible a escala

Atributos en escalas grandes dominan la distancia. Normaliza siempre antes de usar k-NN.

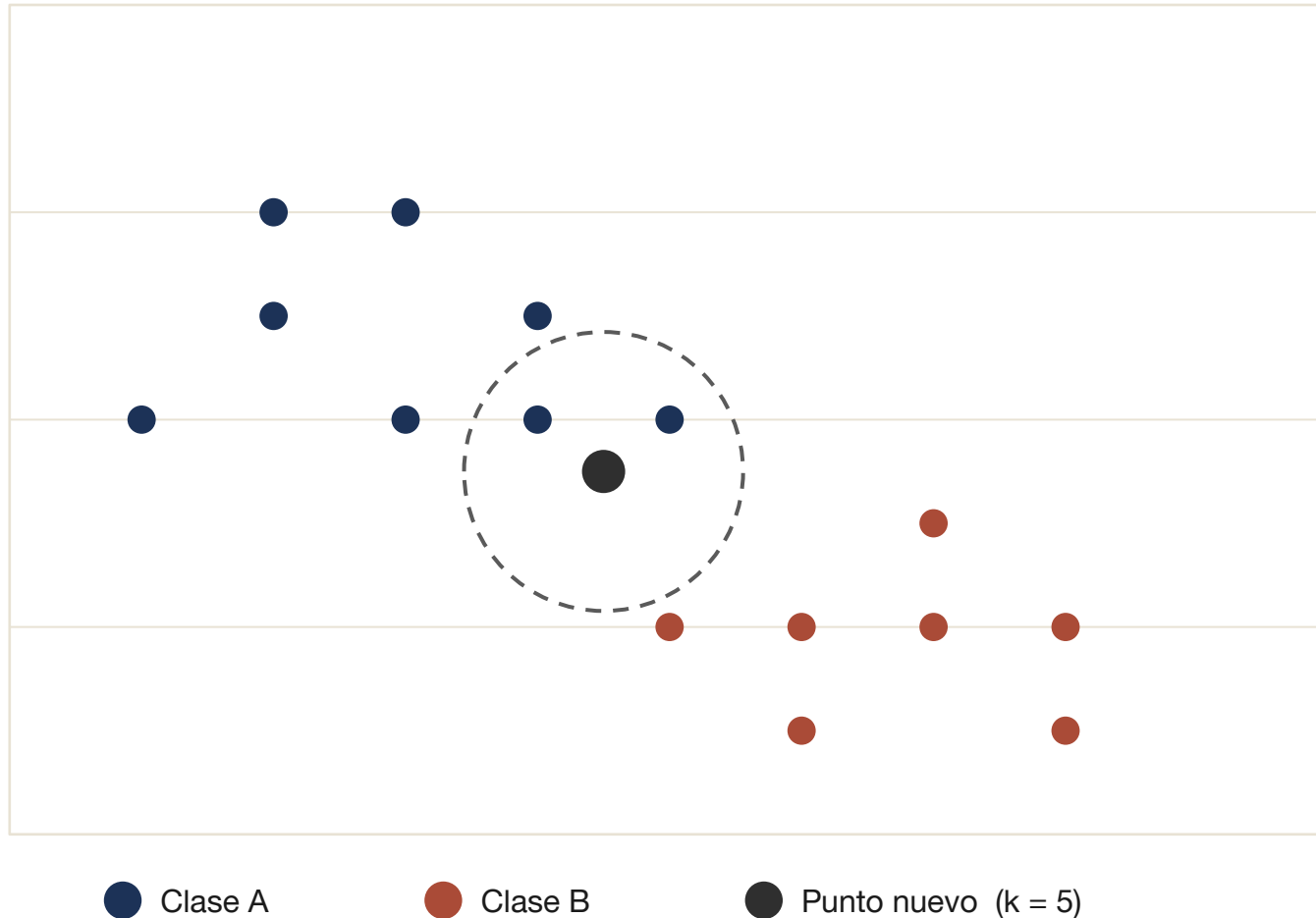


# Cómo elegir k

| Valor de k                | Frontera        | Riesgo                   | Cuándo conviene                    |
|---------------------------|-----------------|--------------------------|------------------------------------|
| <b>k = 1</b>              | Muy irregular   | Overfitting fuerte       | Datos perfectamente limpios (raro) |
| <b>k pequeño (3–5)</b>    | Local, flexible | Sensible al ruido        | Frontera realmente compleja        |
| <b>k mediano (15–25)</b>  | Suave           | Equilibrio bias–varianza | Punto de partida razonable         |
| <b>k grande (&gt; 50)</b> | Muy suave       | Underfitting             | Datasets grandes y poco ruido      |

*Regla práctica: prueba varios k con validación cruzada y elige el que minimice el error.*

# Clasificar un punto nuevo

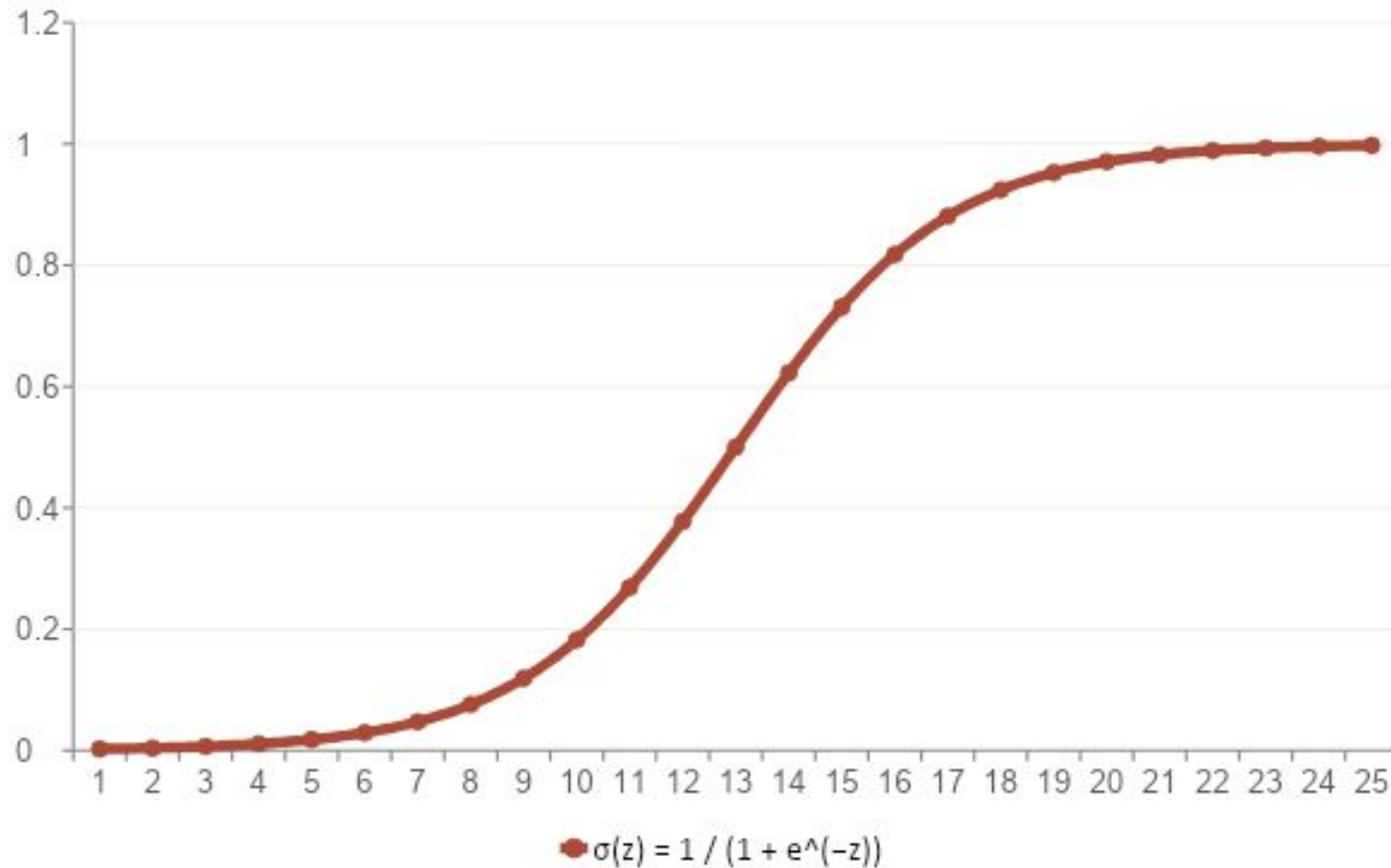


## ¿Cómo decide?

1. Mide distancia del nuevo a cada punto del dataset.
2. Toma los  $k$  más cercanos (aquí  $k = 5$ ).
3. Cuenta a qué clase pertenece cada vecino.
4. Predice la clase mayoritaria.

**Voto:** 3 A + 2 B → **predice A**

# La sigmoide como puente probabilístico



## La idea:

Aplica la regresión lineal y luego comprime su salida al rango  $(0, 1)$  con la función sigmoide.

$$\hat{y} = \sigma(\mathbf{w}^T \mathbf{x} + b)$$

*La salida ya no es un número sin límite — es una probabilidad.*

# De probabilidad a etiqueta

si  $\sigma(w^T x + b) > 0.5 \rightarrow$  clase 1   |   si no  $\rightarrow$  clase 0

## Frontera lineal

La superficie de decisión es un hiperplano. Interpretable y rápido de evaluar.

## Probabilidad calibrada

Te dice qué tan seguro está, no solo qué clase. Útil para priorizar acciones.

## Costo de error ajustable

Moviendo el umbral 0.5 controlas el trade-off entre precisión y recall.

# ¿Qué significan los pesos?

odds ratio =  $e^{w_j}$

Si el atributo  $x_j$  aumenta en una unidad, las odds de la clase 1 se multiplican por  $e^{w_j}$ .

## Ejemplo: modelo de churn sobre Telco

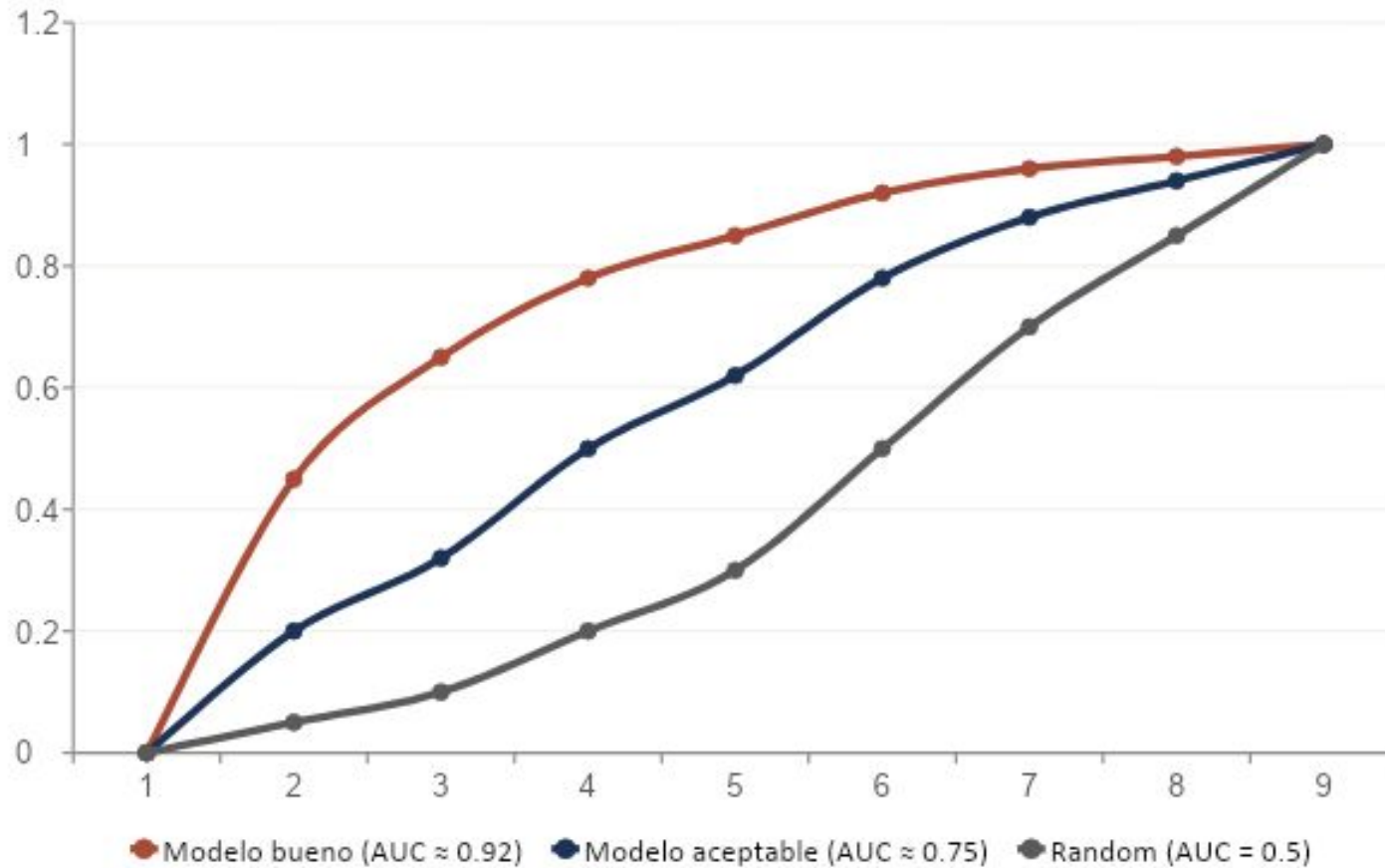
| Atributo              | peso $w_j$ | $e^{w_j}$ | Lectura del coeficiente                                |
|-----------------------|------------|-----------|--------------------------------------------------------|
| tenure                | -0.05      | 0.95      | cada mes adicional reduce ~5% las odds de churn        |
| Contract_MonthToMonth | +1.80      | 6.05      | contrato mes-a-mes multiplica por 6 las odds de churn  |
| OnlineSecurity_Yes    | -0.45      | 0.64      | tener seguridad en línea reduce ~36% las odds de churn |

# Matriz de confusión

|      |          | PREDICHO                               |                                        |
|------|----------|----------------------------------------|----------------------------------------|
|      |          | Positivo                               | Negativo                               |
| REAL | Positivo | <b>TP</b><br><i>Verdadero positivo</i> | <b>FN</b><br><i>Falso negativo</i>     |
|      | Negativo | <b>FP</b><br><i>Falso positivo</i>     | <b>TN</b><br><i>Verdadero negativo</i> |

|                  |                               |                                                        |
|------------------|-------------------------------|--------------------------------------------------------|
| <b>Accuracy</b>  | $(TP + TN) / \text{total}$    | <i>Engaña con clases desbalanceadas.</i>               |
| <b>Precision</b> | $TP / (TP + FP)$              | <i>De los que predije positivos, ¿cuántos lo eran?</i> |
| <b>Recall</b>    | $TP / (TP + FN)$              | <i>De los positivos reales, ¿cuántos capté?</i>        |
| <b>F1</b>        | $2 \cdot P \cdot R / (P + R)$ | <i>Media armónica de precision y recall.</i>           |

# Curva ROC y AUC



## Cómo leerla:

Eje X: FPR (1 - especificidad)

Eje Y: TPR (sensibilidad)

Cuanto más arriba a la izquierda esté la curva, mejor el modelo.

## AUC (Area Under Curve)

Un único número entre 0 y 1. AUC = 1 → perfecto. AUC = 0.5 → equivalente a azar.

# Validación cruzada (k-fold)

*Divide el dataset en  $k$  partes. Entrena  $k$  veces, dejando una parte distinta para test cada vez.*

|             |       |       |       |       |       |
|-------------|-------|-------|-------|-------|-------|
| Iteración 1 | TEST  | TRAIN | TRAIN | TRAIN | TRAIN |
| Iteración 2 | TRAIN | TEST  | TRAIN | TRAIN | TRAIN |
| Iteración 3 | TRAIN | TRAIN | TEST  | TRAIN | TRAIN |
| Iteración 4 | TRAIN | TRAIN | TRAIN | TEST  | TRAIN |
| Iteración 5 | TRAIN | TRAIN | TRAIN | TRAIN | TEST  |

*Reporta el promedio (y la desviación) de las  $k$  evaluaciones. Es una estimación mucho más honesta que un solo train/test split.*



# Manos a la obra sobre Telco Churn

## Lo que tienes que hacer:

1

### Regresión

Predice TotalCharges (continuo). Aplica al menos UN modelo de regresión visto en clase.

2

### Clasificación

Predice Churn (sí/no). Aplica al menos DOS modelos de clasificación distintos y compáralos.

3

### Insights

Cierra el notebook con 3 a 5 hallazgos sobre el dataset que solo se pueden ver con tus modelos.

# Entregables y rúbrica

## Entregable

**Formato:**

Un único archivo .ipynb

**Modalidad:**

Individual

**Dataset:**

Telco Customer Churn (el de la clase pasada)

**Defensa:**

Oral de 5 minutos. Sin defensa, no hay nota.

**Fecha:**

*Por definir (se anuncia esta semana)*

## Rúbrica (sobre 100)

|                          |                                         |     |
|--------------------------|-----------------------------------------|-----|
| Preprocessing            | Manejo de missing, encoding, escalado.  | 15% |
| Modelo de regresión      | Entrenamiento y reporte de métricas.    | 20% |
| Modelos de clasificación | Al menos dos, comparados honestamente.  | 25% |
| Evaluación               | Matriz de confusión, ROC, CV.           | 15% |
| Insights y discusión     | 3–5 hallazgos sustentados.              | 15% |
| Defensa oral             | Explicas tu propio código en 3 minutos. | 10% |

Para la próxima clase

# Lleva tu .ipynb a la siguiente sesión.

**Lectura obligatoria**

*Han, Pei & Tong — Data Mining: Concepts and Techniques (4ª ed.), Capítulo 6 completo (Classification: Basic Concepts and Methods).*

**Siguiente sesión**

*Examen*

*Preguntas, dudas, tropezones: tráelos a clase o al canal del curso.*